

EXAM POV NOTES (Q&A FORMAT)

10 Marks Questions

Q1. Explain the Wumpus World problem in detail. Describe the environment, percepts, actions, and how an agent navigates safely through the grid. Include the rules about stench, breeze, pits, and the wumpus.

The Wumpus World is a classic AI problem demonstrating knowledge-based agents operating in partially observable, deterministic environments with logical reasoning for safe navigation.

- **Environment Description:** The Wumpus World consists of a grid of squares (typically 4×4). The agent starts in the bottom-left square [1,1]. The environment contains one wumpus (a monster), one gold piece, and multiple pits distributed across the grid.
- **Percepts - Stench:** The agent perceives a stench in squares that contain the wumpus or are adjacent to the wumpus. Adjacent means directly left, right, top, or bottom (not diagonal). This warning helps the agent avoid the wumpus location.
- **Percepts - Breeze:** The agent perceives a breeze in squares adjacent to pits. Like stench, adjacency is defined as the four cardinal directions. Multiple breezes can indicate proximity to multiple pits, requiring careful logical deduction.
- **Agent Death Conditions:** The agent dies immediately if it enters a square containing a pit or the living wumpus. This makes exploration dangerous and requires the agent to reason about safe squares before moving.
- **Agent Actions - Movement:** The agent can move forward, turn left 90°, or turn right 90°. The agent maintains a direction (North, South, East, West) and moves one square at a time in the current direction.
- **Agent Actions - Shooting:** The agent has exactly one arrow that can be shot in a straight line in the current direction. If the arrow hits the wumpus, the wumpus dies and becomes safe to pass. The agent must deduce the wumpus location before shooting.
- **Objective and Strategy:** The agent's goal is to find the gold, return to the starting square [1,1], and climb out. The agent must use logical inference from percepts to determine safe squares and infer locations of hazards without direct observation.
- **Knowledge-Based Reasoning:** The agent maintains a knowledge base of facts: "If breeze in [2,1] and no breeze in [1,1], then pit must be in [2,2] or [3,1]." By combining multiple percepts, the agent eliminates possibilities and identifies safe paths.
- **Example Navigation Scenario:** Starting at [1,1] with no percepts means adjacent squares [2,1] and [1,2] are safe. Moving to [2,1] and sensing breeze indicates pit in [2,2], [3,1], or [1,1] (but [1,1] is known safe). Agent reasons through each move using propositional or first-order logic.
- **Challenge and Complexity:** The Wumpus World demonstrates key AI challenges: partial observability (can't see hazards directly), reasoning under uncertainty (multiple possible hazard configurations), planning (finding optimal path to gold), and resource constraints (one arrow).

Q2. Define First-Order Logic with its complete syntax including sentences, atomic sentences, terms, connectives, and quantifiers. Explain how it differs from Propositional Logic with examples.

First-Order Logic (FOL) extends propositional logic by introducing objects, relations, and quantification, enabling representation of complex knowledge about the world.

- **Sentence Structure:** Sentence $\rightarrow$ AtomicSentence | Sentence Connective Sentence | Quantifier Variable,... Sentence | $\neg$ Sentence | (Sentence). This recursive definition allows building complex nested expressions from atomic components.

- **Atomic Sentences:** AtomicSentence $\rightarrow$ Predicate(Term,...) | Term = Term. Examples: Likes(John, IceCream), Father(x, y), x = 5. Atomic sentences express basic facts or relationships between objects using predicates.
- **Terms Definition:** Term $\rightarrow$ Function(Term,...) | Constant | Variable. Terms refer to objects in the domain. Constants like "John" or "5" denote specific objects; variables like x, y can be bound to any object.
- **Constants and Variables:** Constants $\rightarrow$ A | 5 | Kolkata (specific objects). Variables $\rightarrow$ a | x | s (placeholders for objects). Example: Likes(John, x) means John likes some object x, where x can be substituted with any constant.
- **Predicates:** Predicate $\rightarrow$ Before | HasColor | Raining | Loves. Predicates express properties (unary) or relations (n-ary) between objects. Example: Raining() is 0-ary property; Loves(x,y) is binary relation.
- **Functions:** Function $\rightarrow$ Mother | Cosine | HeadOfList. Functions map objects to objects. Example: Mother(John) returns John's mother; LeftLeg(John) returns John's left leg object.
- **Connectives:** Connective $\rightarrow$ $\wedge$ | $\vee$ | $\Rightarrow$ | $\Leftrightarrow$ | $\neg$. These logical operators (AND, OR, IMPLIES, IFF, NOT) combine sentences exactly as in propositional logic, maintaining same truth tables.
- **Quantifiers:** Quantifier $\rightarrow$ $\forall$ (universal) | $\exists$ (existential). $\forall x$ expresses "for all x" while $\exists x$ means "there exists an x". Example: $\forall x \text{ King}(x) \Rightarrow \text{Person}(x)$ means "all kings are persons."
- **Difference from Propositional Logic - Expressiveness:** Propositional logic uses atomic propositions (P, Q, R) representing complete facts. FOL breaks facts into objects and relations. "John loves Mary" is one proposition in PL but Loves(John, Mary) in FOL, allowing generalization.
- **Difference - Quantification Power:** FOL can express "all students take history" as $\forall x \text{ Student}(x) \Rightarrow \text{Takes}(x, \text{History})$. Propositional logic requires separate propositions for each student: StudentJohnTakesHistory $\wedge$ StudentMaryTakesHistory $\wedge$... becoming impractical for large domains.

Q3. Explain the Resolution inference technique in First-Order Logic. Include the process of converting statements to Conjunctive Normal Form (CNF), unification, and demonstrate with the "Gaul Traitorix" criminal problem from the document.

Resolution is a complete inference rule for FOL that operates on clauses in CNF, using unification to combine complementary literals and derive conclusions.

- **Resolution Principle:** Resolution rule states: Given clauses $(\alpha \vee \beta)$ and $(\neg\alpha \vee \gamma)$, we can infer $(\beta \vee \gamma)$. This eliminates complementary literals α and $\neg\alpha$. In FOL, we must first unify the literals to make them identical before resolving.
- **Unit Resolution:** Special case where one clause is a single literal. From clause α and clause $(\neg\alpha \vee \beta)$, we infer β .
Example: From Happy(John) and $(\neg\text{Happy}(\text{John}) \vee \text{Laughs}(\text{John}))$, infer Laughs(John).
- **Conjunctive Normal Form (CNF):** Sentence expressed as conjunction of clauses: $(L_1 \vee L_2 \vee \dots) \wedge (M_1 \vee M_2 \vee \dots) \wedge \dots$. Each clause is disjunction of literals.
Example: $(P \vee Q) \wedge (\neg P \vee R) \wedge (\neg Q \vee \neg R)$.
- **Converting to CNF - Step 1 (Eliminate Implications):** Replace $A \Rightarrow B$ with $\neg A \vee B$.
Replace $A \Leftrightarrow B$ with $(\neg A \vee B) \wedge (A \vee \neg B)$. This removes implication and biconditional connectives.
- **Converting to CNF - Step 2 (Move Negation Inward):** Apply De Morgan's laws:
 $\neg(A \wedge B)$ becomes $(\neg A \vee \neg B)$; $\neg(A \vee B)$ becomes $(\neg A \wedge \neg B)$. Move negations to literals using $\neg\neg A = A$, $\neg\forall x$ becomes $\exists x\neg$, and $\neg\exists x$ becomes $\forall x\neg$.

- **Converting to CNF - Step 3 (Standardize Variables):** Rename variables so each quantifier has unique variable name. $\forall x P(x) \vee \forall x Q(x)$ becomes $\forall x P(x) \vee \forall y Q(y)$ to avoid confusion.
- **Converting to CNF - Step 4 (Skolemization):** Eliminate existential quantifiers by replacing existentially quantified variables with Skolem functions. $\forall x \exists y \text{Loves}(x,y)$ becomes $\forall x \text{Loves}(x, F(x))$ where F is Skolem function.
- **Converting to CNF - Step 5 (Distribute):** Distribute $\vee$ over $\wedge$ using $(A \vee (B \wedge C)) = (A \vee B) \wedge (A \vee C)$. Continue until formula is in CNF form. Drop universal quantifiers (all variables implicitly universally quantified).
- **Traitorix Problem - Knowledge Base:**
 - It is crime for Gaul to sell potion formulas to hostile nations:
 $\forall x,y,z \text{Gaul}(x) \wedge \text{Formula}(y) \wedge \text{Nation}(z) \wedge \text{Hostile}(z) \wedge \text{Sells}(x,y,z) \Rightarrow \text{Criminal}(x)$
 - Rome is enemy of Gaul: $\text{Enemy}(\text{Rome}, \text{Gaul}) \wedge \text{Hostile}(\text{Rome})$
 - Rome acquired formulas sold by Traitorix:
 $\forall y \text{OwnedBy}(y, \text{Rome}) \Rightarrow \text{Sells}(\text{Traitorix}, y, \text{Rome})$
 - Traitorix is Gaul: $\text{Gaul}(\text{Traitorix})$
- **Traitorix Problem - Conversion to CNF:** Convert KB to CNF clauses:
 - $\neg \text{Gaul}(x) \vee \neg \text{Formula}(y) \vee \neg \text{Nation}(z) \vee \neg \text{Hostile}(z) \vee \neg \text{Sells}(x,y,z) \vee \text{Criminal}(x)$
 - $\text{Hostile}(\text{Rome})$
 - $\neg \text{OwnedBy}(y, \text{Rome}) \vee \text{Sells}(\text{Traitorix}, y, \text{Rome})$
 - $\text{Gaul}(\text{Traitorix})$
 - Add: $\text{Formula}(F_1)$ [some formula], $\text{OwnedBy}(F_1, \text{Rome})$

Q4. Compare and contrast Forward Chaining and Backward Chaining inference methods. Explain their properties, approaches (data-driven vs goal-driven), and applications with examples.

Forward and backward chaining are two fundamental inference strategies for rule-based systems, differing in direction of reasoning and problem-solving approach.

- **Forward Chaining Definition:** Forward chaining starts from known facts in the knowledge base and applies inference rules (primarily Modus Ponens) in forward direction to derive new facts until goal is reached or no more inferences possible.
- **Forward Chaining Process:** Algorithm repeatedly finds all rules whose premises are satisfied by current facts, adds conclusions to knowledge base, and continues. Example: Given "Socrates is man" and "All men are mortal" $\Rightarrow$ infer "Socrates is mortal."
- **Forward Chaining Direction:** Bottom-up or data-driven approach, moving from initial state toward goal state. Starts with evidence (data) and chains forward through rules to discover what conclusions follow from available information.
- **Forward Chaining Properties:** Data-driven strategy means reasoning is triggered by arrival of new data/facts. Explores all possible inferences from known facts. May derive many irrelevant facts before reaching goal. Efficient when many goals possible from few facts.
- **Forward Chaining Applications:** Commonly used in expert systems (like CLIPS), business rule engines, production systems, monitoring systems, and reactive agents. Ideal when system must respond to data inputs and trigger appropriate actions.
- **Backward Chaining Definition:** Backward chaining starts with goal (query) and works backward through rules to find known facts supporting the goal. Recursively breaks goal into sub-goals until reaching facts in knowledge base.
- **Backward Chaining Process:** Algorithm asks "To prove goal G , what facts needed?" Finds rules concluding G , treats their premises as new sub-goals, and recursively proves each. Example: Goal "Socrates is mortal" $\leftarrow$ need "Socrates is man" $\leftarrow$ check KB for this fact.

- **Backward Chaining Direction:** Top-down or goal-driven approach, moving from goal state back to initial state. Starts with hypothesis and chains backward through rules to verify if hypothesis supported by known facts.
- **Backward Chaining Properties:** Goal-driven means list of goals determines which rules selected. Uses depth-first search strategy typically. Avoids exploring irrelevant facts. More focused than forward chaining. Efficient when few goals need verification from many possible facts.
- **Backward Chaining Applications:** Used in automated theorem proving, logic programming (Prolog), proof assistants, game theory, diagnostic systems, and query-answering systems. Ideal when specific query must be answered from large knowledge base.

5 Marks Questions

Q5. Solve the Unicorn problem using Propositional Logic: "If the unicorn is mythical, then it is immortal, but if it is not mythical, then it is a mortal mammal. If the unicorn is either immortal or a mammal, then it is horned. The unicorn is magical if it is horned." Prove whether the unicorn is horned and magical.

The Unicorn problem demonstrates logical inference using propositional logic to derive conclusions from conditional statements about mythical creatures.

- **Define Propositions:** Umyth = Unicorn is mythical; Umort = Unicorn is mortal; Umam = Unicorn is mammal; Uhorn = Unicorn is horned; Umag = Unicorn is magical. These atomic propositions represent basic facts about the unicorn.
- **Statement 1 Translation:** "If unicorn is mythical then immortal, but if not mythical then mortal mammal" = $(Umyth \Rightarrow \neg Umort) \wedge (\neg Umyth \Rightarrow (Umort \wedge Umam))$. This covers both cases of unicorn being mythical or not.
- **Statement 2 Translation:** "If unicorn is either immortal or mammal, then it is horned" = $((\neg Umort \vee Umam) \Rightarrow Uhorn)$. Immortal means not mortal, so $\neg Umort$. The condition is satisfied if either condition holds.
- **Statement 3 Translation:** "Unicorn is magical if it is horned" = $(Uhorn \Rightarrow Umag)$. This is straightforward implication from horned to magical property.
- **Proving Unicorn is Horned - Case Analysis:** Consider two cases:
Case 1: Assume Umyth is true. Then $\neg Umort$ is true (immortal). From $(\neg Umort \vee Umam)$, the condition is satisfied. Therefore Uhorn is true by Statement 2.
Case 2: Assume Umyth is false ($\neg Umyth$). Then $(Umort \wedge Umam)$ is true by Statement 1. Since Umam is true, $(\neg Umort \vee Umam)$ is satisfied, so Uhorn is true. In both cases, Uhorn must be true. Conclusion: The unicorn is definitely horned.
- **Proving Unicorn is Magical:** From previous proof, we established Uhorn is true. By Statement 3, $Uhorn \Rightarrow Umag$. Since Uhorn is true, we can apply Modus Ponens to conclude Umag is true. Therefore, the unicorn is magical.
- **Attempting to Prove Mythical:** Can we prove Umyth? From our case analysis, the unicorn is horned whether mythical or not. We have no statement that concludes Umyth. Statements only have Umyth as premise, never conclusion. Therefore, we cannot determine if unicorn is mythical - it remains unknown.
- **Summary of Conclusions:** Proven TRUE: Uhorn (horned), Umag (magical).
Unknown: Umyth (mythical), Umort (mortal), Umam (mammal). The logical structure guarantees certain properties regardless of mythical status, demonstrating power of propositional inference.

Q6. Explain the different types of logic systems mentioned in the document. Compare Propositional Logic, First-Order Logic, Temporal Logic, Probability Theory, and Fuzzy Logic based on what exists and agent beliefs.

Different logic systems provide varying levels of expressiveness and handle uncertainty differently, suited for different reasoning tasks in artificial intelligence.

- **Propositional Logic:** Represents facts as atomic propositions. What exists: Only facts (true/false statements). Agent belief: T (true), F (false), or Unknown. Example: "It is raining" = P. Cannot express relationships between objects or reason about time or uncertainty degrees.
- **First-Order Logic (FOL):** Extends propositional logic with objects, relations, functions, and quantification. What exists: Facts, Objects, Relations between objects. Agent belief: T, F, or Unknown. Example: $\forall x \text{ Student}(x) \Rightarrow \text{Person}(x)$. More expressive than PL, can represent complex world knowledge.
- **Temporal Logic:** Extends FOL by adding temporal dimension. What exists: Facts, Objects, Relations, and Time points or intervals. Agent belief: T, F, or Unknown at specific times. Example: "Always Eventually(Goal)" or "Until(Safe, Rescue)". Can reason about sequence of events, planning, and temporal constraints.
- **Probability Theory:** Handles uncertain knowledge using numerical probabilities. What exists: Facts (events). Agent belief: Degree of belief expressed as real number in $[0,1]$ range, where 0 = impossible, 1 = certain, 0.7 = 70% confident. Example: $P(\text{Rain} \mid \text{CloudySky}) = 0.8$. Follows probability axioms and calculus.
- **Fuzzy Logic:** Handles vagueness and partial truth rather than uncertainty. What exists: Facts with degrees of truth. Agent belief: Degree of belief in $[0,1]$, but represents how true something is, not confidence. Example: "Temperature is hot" might be 0.7 true (somewhat hot). Different from probability - 0.7 hot doesn't mean 70% chance of being hot.
- **Key Differences - Representation:** PL and FOL are boolean (crisp true/false). Temporal adds time dimension. Probability and Fuzzy use $[0,1]$ scale but for different purposes (uncertainty vs. vagueness). FOL most structurally expressive for object relationships.
- **Key Differences - Reasoning:** PL/FOL use deductive reasoning (certain conclusions from certain premises). Probability uses probabilistic inference (Bayes theorem, conditioning). Fuzzy uses fuzzy inference rules with fuzzy membership functions. Temporal uses modal operators (Always, Eventually, Until).
- **Applications and Trade-offs:** PL: Simple, fast, limited expressiveness. FOL: Powerful representation, computationally expensive. Temporal: Planning, verification of systems. Probability: Machine learning, decision theory, uncertain environments. Fuzzy: Control systems, approximate reasoning. Choice depends on problem domain and computational constraints.

Q7. Describe the three inference rules for First-Order Logic: Universal Elimination, Existential Elimination, and Existential Introduction. Provide examples for each.

These inference rules allow us to manipulate quantified statements in First-Order Logic, connecting universal/existential statements with specific instances.

- **Universal Elimination ($\forall$ -Elimination) Definition:** Also called Universal Instantiation. From $\forall x P(x)$, we can infer $P(c)$ for any constant c in the domain. This removes universal quantifier by substituting a specific ground term (constant) for the variable.

- **Universal Elimination Example:** Given $\forall x \text{ Likes}(x, \text{IceCream})$ meaning "everyone likes ice cream," we can apply substitution $\{x/\text{Einstein}\}$ to infer $\text{Likes}(\text{Einstein}, \text{IceCream})$. Similarly, can substitute any person: $\text{Likes}(\text{Mary}, \text{IceCream})$, $\text{Likes}(\text{John}, \text{IceCream})$, etc.
- **Universal Elimination Constraint:** The substitution must use a ground term (constant), not another variable or complex expression with new variables. Cannot substitute x with y to get $\forall x P(x) \rightarrow P(y)$ with free y . Ground term ensures we're reasoning about specific object in domain.
- **Existential Elimination ($\exists$ -Elimination) Definition:** Also called Existential Instantiation. From $\exists x P(x)$, we can infer $P(c)$ where c is a new constant (Skolem constant) that doesn't appear elsewhere in knowledge base. This "witnesses" the existence claim.
- **Existential Elimination Example:** From $\exists x \text{ Likes}(x, \text{IceCream})$ meaning "someone likes ice cream," we may infer $\text{Likes}(\text{Man}, \text{IceCream})$ where Man is a new symbol. We don't know who this person is, so we introduce fresh constant to represent them. Man must not appear elsewhere in KB to avoid incorrect assumptions.
- **Existential Elimination Constraint:** The new constant must be fresh/unique and not appear elsewhere in knowledge base. This prevents us from incorrectly assuming the existentially quantified object is same as some already-known object. Maintains logical soundness by treating existence witness as distinct unknown entity.
- **Existential Introduction ($\exists$ -Introduction) Definition:** From $P(c)$ for specific constant c , we can infer $\exists x P(x)$. If property holds for specific instance, we can generalize to claim that something exists with that property. This is opposite direction of existential elimination.
- **Existential Introduction Example:** From specific fact $\text{Likes}(\text{Monalisa}, \text{IceCream})$ meaning "Monalisa likes ice cream," we can infer $\exists x \text{ Likes}(x, \text{IceCream})$ meaning "there exists someone who likes ice cream." This is always valid - if true for specific case, certainly true that some case exists.

Q8. What is Unification in First-Order Logic? Explain the process with examples and define Most General Unifier (MGU). Demonstrate with $\text{UNIFY}\{\text{King}(x), \text{King}(\text{John})\}$.

Unification is fundamental to automated reasoning in FOL, making different logical expressions identical through variable substitutions for inference rules like resolution.

- **Unification Definition:** Unification is process of finding substitution θ that makes two different logical atomic expressions identical. Takes two literals as input and returns substitution making them syntactically equal, or fails if no such substitution exists.
- **Formal Notation:** Let Ψ_1 and Ψ_2 be two atomic sentences. A unifier θ satisfies $\Psi_1\theta = \Psi_2\theta$ (applying θ to both makes them identical). Expressed as $\text{UNIFY}(\Psi_1, \Psi_2) = \theta$ or FAIL.
- **Simple Example - $\text{UNIFY}\{\text{King}(x), \text{King}(\text{John})\}$:** $\Psi_1 = \text{King}(x)$, $\Psi_2 = \text{King}(\text{John})$. Substitution $\theta = \{x/\text{John}\}$ replaces x with John . Applying substitution: $\text{King}(x)\theta = \text{King}(\text{John})$, $\text{King}(\text{John})\theta = \text{King}(\text{John})$. Since both become $\text{King}(\text{John})$, they are unified. Therefore UNIFY returns $\{x/\text{John}\}$.
- **Most General Unifier (MGU) Definition:** Among all possible unifiers for two expressions, MGU is most general one that makes fewest commitments. MGU can be specialized to produce any other unifier through additional substitutions. Makes minimal bindings necessary for unification.
- **MGU Example:** For $\text{UNIFY}\{P(x, y), P(\text{John}, z)\}$, possible unifiers include $\{x/\text{John}, y/\text{John}, z/\text{John}\}$ or $\{x/\text{John}, z/y\}$. MGU is $\{x/\text{John}, z/y\}$ because it's more general - doesn't commit y and z to specific value unnecessarily. Can specialize MGU by adding $\{x/\text{John}, z/y\}$ later if needed.

- **Unification Algorithm Process:** Compare predicates - must match. Recursively compare arguments left to right. If argument is variable, bind it to corresponding term (if variable doesn't occur in that term - occurs check). If both constants, must be identical. Build substitution by accumulating bindings. Fail if contradiction found.
- **Complex Unification Example:** UNIFY{Loves(x, Father(x)), Loves(John, y)}. First arguments: x and John $\rightarrow$ {John/x}. Second arguments: Father(x) and y $\rightarrow$ {Father(John)/y} after applying {John/x}. Result: $\theta = \{John/x, Father(John)/y\}$. Both expressions become Loves(John, Father(John)).
- **Occurs Check:** Critical part of unification - prevent infinite structures. Cannot unify x with F(x) because substitution {F(x)/x} creates F(F(F(...))) infinite term. UNIFY{P(x), P(F(x))} must FAIL. Occurs check verifies variable doesn't appear in term being substituted for it.

Q9. Explain Horn sentences and their significance in inference. Why does restricting to Horn sentences give us a polynomial-time procedure compared to the general NP-complete inference problem?

Horn clauses enable efficient inference through forward and backward chaining, providing tractable reasoning for practical AI systems despite FOL inference being NP-complete generally.

- **General Inference Complexity:** In general, inference problem in propositional and first-order logic is NP-complete (Cook's Theorem). Determining if $KB \models \alpha$ requires exponential time in worst case. For n propositions, may need to check 2^n possible truth assignments. Makes general logical inference computationally intractable for large knowledge bases.
- **Horn Sentence Definition:** Horn clause is disjunction of literals with at most one positive literal. Equivalently, can be written as implication: $P_1 \wedge P_2 \wedge \dots \wedge P_n \Rightarrow Q$, where $P_1 \dots P_n$ (premises) and Q (conclusion) are positive literals. Special cases: Q (fact with no premises) or $P_1 \wedge P_2 \wedge \dots \wedge P_n \Rightarrow \text{False}$ (integrity constraint).
- **Horn Clause Forms:** Definite clause: exactly one positive literal ($P_1 \wedge P_2 \Rightarrow Q$). Fact: single positive literal with no premises (Q). Goal clause: no positive literals ($P_1 \wedge P_2 \Rightarrow \text{False}$, used in query). Horn clauses exclude disjunctions in conclusion (cannot have $P \Rightarrow (Q \vee R)$).
- **Modus Ponens Applicability:** Horn clauses perfectly match Modus Ponens inference rule: Given $P_1 \wedge P_2 \wedge \dots \wedge P_n \Rightarrow Q$ and facts $P_1, P_2, \dots, P_n$ in KB, infer Q. This is sound, complete for Horn clauses, and computationally efficient - just pattern matching.
- **Forward Chaining on Horn Clauses:** Start with known facts. Find all rules whose premises satisfied by current facts. Add their conclusions to fact base. Repeat until no new facts derived or goal reached. Each rule fires at most once. If m rules and n symbols, at most $O(mn)$ time complexity - polynomial!
- **Polynomial Time Reasoning:** Key insight: In Horn clause KB with m rules and n symbols, forward chaining terminates in $O(mn)$ time. Each rule examined at most n times. Each fact added at most once. No backtracking or exponential search needed. Makes Horn clause reasoning tractable for expert systems.
- **Restrictions and Trade-offs:** Horn clauses cannot express arbitrary disjunctions ($P \vee Q \vee R$). Cannot say "Either it rains or it snows." Cannot represent XOR directly. Despite limitations, many practical problems expressible as Horn clauses: production rules, taxonomies, most expert system rules, Prolog programs.
- **Practical Importance:** Most expert systems (CLIPS, JESS), Prolog programming language, and many AI planning systems use Horn clause representation. Trade-off between expressiveness and efficiency favors Horn clauses for real-world applications where performance matters. Logic programming's success relies on Horn clause tractability.

Q10. Define Resolution inference rule and Unit Resolution. Explain how they differ and provide examples of both.

Resolution and Unit Resolution are refutation-based inference rules that form the foundation of automated theorem proving in propositional and first-order logic.

- **Resolution Rule Definition:** Resolution is inference rule operating on clauses (disjunctions of literals). Given two clauses containing complementary literals (α and $\neg\alpha$), resolution produces new clause with those literals removed.
Formal: From $(\alpha \vee \beta)$ and $(\neg\alpha \vee \gamma)$, infer $(\beta \vee \gamma)$. This is called resolvent.
- **Resolution Example:** Given clauses $(P \vee Q)$ and $(\neg P \vee R)$, literals P and $\neg P$ are complementary. Resolution produces $(Q \vee R)$. Another example: $(A \vee B \vee C)$ and $(\neg B \vee D \vee E)$ resolve on B to produce $(A \vee C \vee D \vee E)$.
- **Resolution Completeness:** Resolution is refutation complete - if $KB \cup \{\neg\alpha\}$ is unsatisfiable, resolution will derive empty clause (contradiction). To prove $KB \models \alpha$, add $\neg\alpha$ to KB and apply resolution repeatedly. If empty clause derived, α is proven. This proof by contradiction is foundation of resolution theorem proving.
- **Unit Resolution Definition:** Special case of resolution where one of the clauses is unit clause (single literal). From unit clause α and clause $(\neg\alpha \vee \beta)$, infer β . Since one clause contains only one literal, resolvent is simpler than general resolution.
- **Unit Resolution Example:** From unit clause P and clause $(\neg P \vee Q \vee R)$, resolve on P to get $(Q \vee R)$. From $\text{Happy}(\text{John})$ and $(\neg\text{Happy}(\text{John}) \vee \text{Smiles}(\text{John}))$, infer $\text{Smiles}(\text{John})$. Unit resolution is computationally simpler because one clause has single literal.
- **Key Differences:** General resolution: both clauses can have multiple literals, resulting resolvent can be large. Unit resolution: one clause is single literal, producing simpler resolvents. Unit resolution is subset of general resolution but forms basis for efficient forward chaining algorithms.
- **Resolution in FOL:** In first-order logic, must unify complementary literals before resolving. From $\neg\text{King}(x) \vee \text{Person}(x)$ and $\text{King}(\text{John})$, first unify $\text{King}(x)$ with $\text{King}(\text{John})$ using $\theta = \{\text{John}/x\}$, producing $\neg\text{King}(\text{John}) \vee \text{Person}(\text{John})$, then resolve to get $\text{Person}(\text{John})$.
- **Practical Application:** Resolution is complete inference procedure - can derive any logical consequence. Used in automated theorem provers (Prover9, Vampire), logic programming (Prolog uses SLD-resolution variant), and satisfiability solvers. Unit resolution particularly efficient for Horn clause reasoning in forward chaining systems.

2 Marks Questions

Q11. What is a formal logic system? Define syntax, semantics, and proof theory.

- **Formal Logic System:** A formal system for describing states of affairs consisting of syntax (describes how to make sentences), semantics (describes relation between sentences and states of affairs), and proof theory (set of rules for deducing entailments).
- **Three Components:** Syntax defines valid sentence formation rules and grammar. Semantics defines meaning and truth conditions of sentences in models. Proof theory provides inference rules to mechanically derive logical consequences from premises without reference to meaning.

Q12. Write the BNF grammar for Propositional Logic sentences including atoms and complex sentences.

- **Grammar Rules:** $\text{Sentence} \rightarrow \text{Atom} \mid \text{ComplexSentence}$; $\text{Atom} \rightarrow \text{True} \mid \text{False} \mid \text{AP}$ (atomic proposition); $\text{ComplexSentence} \rightarrow (\text{Sentence}) \mid \neg \text{Sentence} \mid \text{Sentence Connective Sentence}$; $\text{Connective} \rightarrow \wedge \mid \vee \mid \Rightarrow \mid \Leftrightarrow$.
- **Structure:** Atoms are basic propositions (P, Q, R) or Boolean constants. Complex sentences built recursively using negation, binary connectives (AND, OR, IMPLIES, IFF), and parentheses for grouping to form arbitrarily nested expressions.

Q13. State and explain Modus Ponens (Implication Elimination) inference rule with an example.

- **Modus Ponens Rule:** From premises $\alpha \Rightarrow \beta$ (implication) and α (antecedent is true), we can infer β (consequent). Also called Implication Elimination - eliminates implication by affirming antecedent.
- **Example:** Given "If it rains, then ground is wet" ($\text{Rain} \Rightarrow \text{Wet}$) and "It is raining" (Rain), we conclude "Ground is wet" (Wet). Foundation for forward chaining and Horn clause reasoning systems.

Q14. What is Conjunctive Normal Form (CNF)? Define clause and unit clause.

- **CNF Definition:** Sentence expressed as conjunction (AND) of clauses: $(L_{11} \vee L_{12} \vee \dots) \wedge (L_{21} \vee L_{22} \vee \dots) \wedge \dots$, where each L is literal. Clause is disjunction (OR) of literals. Example: $(P \vee Q) \wedge (\neg P \vee R)$.
- **Unit Clause:** Clause containing exactly one literal (P, $\neg Q$, Likes(John, Mary)). Represents facts that must be true. Called "unit" because single disjunct - simplest non-empty clause, crucial for unit resolution and forward chaining.

Q15. List the components of First-Order Logic: Constants, Variables, Predicates, and Functions with examples of each.

- **Constants and Variables:** Constants denote specific objects (A, 5, Kolkata, John) - fixed references. Variables are placeholders (a, x, y, s) that can bind to any domain object, used in quantification.
- **Predicates and Functions:** Predicates express properties/relations, return true/false (Before(t_1, t_2), HasColor(obj,color), Student(x)). Functions map objects to objects, return terms (Mother(x), Cosine(angle), HeadOfList(list)).

Q16. Define the two modes in which an inference engine operates.

- **Forward Chaining:** Starts from known facts and applies inference rules forward to derive new facts until goal reached. Data-driven approach triggered by new data arrival. Used when system determines all consequences of available information.
- **Backward Chaining:** Starts from goal and works backward through rules to find supporting facts. Goal-driven approach guided by specific query. Recursively breaks goal into sub-goals until reaching known facts in knowledge base.

Q17. What is the difference between data-driven and goal-driven approaches in inference?

- **Data-Driven (Forward):** Reasoning triggered by available data, starts with facts and derives all conclusions. Bottom-up control from evidence to conclusions. Used when multiple possible goals or system reacts to incoming data.

- **Goal-Driven (Backward):** Reasoning guided by specific goal/query, starts with hypothesis and searches supporting evidence. Top-down control from goal to facts. Used for focused queries from large knowledge base, more efficient for specific questions.

Q18. In the Wumpus World, what happens if the agent enters a square with a pit or wumpus?

What weapon does the agent have?

- **Death Conditions:** Agent dies immediately if enters square containing pit or living wumpus. Death is permanent - cannot recover. Requires logical inference to determine safe squares before moving into unexplored areas.
- **Agent's Weapon:** Agent has exactly one arrow that shoots straight line in current direction. Arrow kills wumpus if hit (agent hears scream). Only one shot available - must deduce wumpus location logically before shooting.

Q19. Express in First-Order Logic: "No person likes a professor unless the professor is smart."

- **FOL Translation:** $\forall x \forall y (Person(x) \wedge Professor(y) \wedge Likes(x,y)) \Rightarrow Smart(y)$. Means for all persons x and professors y, if x likes y, then y must be smart.
- **Meaning:** "Unless" creates conditional - liked professors must be smart. Equivalent to: all liked professors are smart. Can also write contrapositive:
 $\forall x \forall y (Person(x) \wedge Professor(y) \wedge \neg Smart(y)) \Rightarrow \neg Likes(x,y)$.

Q20. What does it mean for the inference problem to be NP-complete? (Reference Cook's Theorem)

- **NP-Complete Inference:** Cook's Theorem proves logical inference is NP-complete - requires exponential time in worst case. For n propositions, may need checking 2^n truth assignments. Makes general logical inference computationally intractable for large knowledge bases.
- **Practical Impact:** Cannot expect algorithm always solving inference quickly. Motivates restricting to tractable subsets like Horn clauses (polynomial-time inference using forward/backward chaining). Modern solvers use heuristics working well on many practical instances despite worst-case complexity.

Q21. Define substitution in the context of unification. Give one example.

- **Substitution Definition:** A substitution θ is finite set of bindings $\{term_1/var_1, term_2/var_2, \dots\}$ where each var is variable replaced by corresponding term. Applying θ to expression E simultaneously replaces all variable occurrences.
- **Example:** Given $\theta = \{John/x, Mary/y\}$ and expression $Loves(x,y)$, applying substitution yields $Loves(x,y)\theta = Loves(John, Mary)$. Substitution enables unification to make expressions identical for inference rules like resolution.

Q22. What is the main advantage of using Resolution as an inference rule in automated reasoning?

- **Completeness and Uniformity:** Resolution is refutation-complete - single inference rule sufficient for all logical reasoning. If KB entails α , resolution on $KB \cup \{\neg\alpha\}$ will derive empty clause. No need for multiple specialized inference rules.
- **Practical Foundation:** Underlies Prolog (SLD-resolution), SAT solvers (DPLL), automated theorem provers (Vampire, E). Enables systematic exhaustive search. Mechanical nature ideal for computer implementation, works naturally with CNF representation allowing preprocessing optimizations.